

Assignment 2: LLMs for Fault Localization With and Without Tests and SBFL

Topics in Software Engineering

Suggested duration: 4 weeks

Required technologies and tools:

- Python
 - Python unittest or pytest
 - OpenRouter API
 - Git
 - Static HTML
-

1. Overview

In this assignment, you will conduct an empirical software engineering study on the use of large language models (LLMs) for fault localization in Python methods.

You will build and evaluate an automated experimental pipeline that compares how well LLMs identify faulty lines in buggy Python methods under different information conditions.

The assignment focuses on:

- empirical software engineering
- automated experimentation
- software testing
- spectrum-based fault localization
- LLM evaluation
- reproducibility engineering
- experimental infrastructure validation

You will:

- construct a benchmark of buggy Python methods based on HumanEval-style tasks
- generate buggy versions using mutation
- execute tests
- compute SBFL rankings using Tarantula (or other formulas)
- compare at least two LLMs through OpenRouter
- evaluate fault-localization accuracy
- validate your own experimental infrastructure
- build a reproducible artifact
- present your work as a professional static HTML website

3. Research Questions

Your work should address the following research questions.

RQ1 — Baseline Fault Localization

How well do LLMs localize faults when given only the buggy method and the intended behavior?

RQ2 — Effect of Test Information

Does providing passing and failing tests improve LLM fault-localization performance?

RQ3 — Effect of SBFL Information

Does providing a spectrum-based fault-localization ranking, such as Tarantula, improve LLM fault-localization performance?

RQ4 — Combining Tests and SBFL

Does combining test information with an SBFL ranking improve performance more than either source alone?

RQ5 — Model Comparison

Do different LLMs perform differently on the same fault-localization task?

3. Dataset Construction

You must construct a benchmark based on HumanEval. Can also use more advanced sources and more complex methods.

Your dataset must include at least:

- 30 methods
- a short natural-language specification for each method
- At least 10 tests for each method
- one buggy version of each method
- a known ground-truth faulty line or small faulty region
- evidence that the original version passes the tests

- evidence that the buggy version fails at least one test

For each selected HumanEval method, you must create a buggy version.

You may use:

- manual mutation
- automatic mutation
- mutation-testing tools
- scripts written by you
- AI-assisted generation of mutants, with manual validation

Examples of suitable mutations include:

- replacing `<` with `<=`
- replacing `&&` with `||`
- changing an arithmetic operator
- changing a boundary condition
- returning the wrong variable
- deleting a condition
- changing a constant
- mishandling empty input
- introducing an off-by-one error

Each mutant must satisfy the following requirements:

- the original method passes all the tests
- the mutated method fails at least one test
- the faulty line or region is known
- the bug is reasonably localizable
- the mutant is not equivalent to the original program
- the method remains small enough to fit clearly in an LLM prompt

Prefer mutants where at least one test passes and at least one test fails, because this makes the passing/failing distinction meaningful for fault localization.

3. LLM Evaluation Setup and Experimental Conditions

3.1 LLM Evaluation Setup

You must compare at least two LLMs.

All LLM calls must be made through the OpenRouter API.

You should record:

- exact OpenRouter model IDs
- date of execution
- decoding parameters such as temperature
- prompt format
- number of repetitions
- errors or rate-limit issues
- total number of valid and invalid responses

The default requirement is one run per prompt.

3.2 Experimental Conditions

For each buggy method and each LLM, you must run four information conditions.

Condition A — Code Only

Input:

- buggy method
- intended behavior

Condition B — Code + Tests

Input:

- buggy method
- intended behavior
- passing/failing tests

Condition C — Code + SBFL

Input:

- buggy method
- intended behavior
- Tarantula ranking

Condition D — Code + Tests + SBFL

Input:

- buggy method
- intended behavior
- tests
- Tarantula ranking

With 30 methods, 2 models, and 4 conditions, the minimum experiment size is:

30 methods × 2 models × 4 conditions = 240 LLM calls

3.3 Prompting Requirements

You should use a fixed prompt template for all models and all methods.

The prompt should change only according to the information condition.

The model should be asked to return structured output, preferably JSON.

Example:

```
{
  "top_1_line": 12,
  "top_3_lines": [12, 14, 9],
  "faulty_region": "loop condition",
  "explanation": "The loop condition may terminate before the last candidate is checked."
}
```

The prompt should explicitly state:

“Do not repair the program. Your task is only to identify the most likely faulty line or small faulty region.”

You must save the raw model outputs, even when they are incorrect or malformed.

4. Spectrum-Based Fault Localization

For each buggy method, you should run the tests and identify:

- passing tests
- failing tests
- expected output
- actual output, where relevant

You should also collect coverage information and compute a Tarantula-style suspiciousness score for executable lines.

For each line s :

$\text{failed}(s)$ = number of failing tests that execute s

$\text{passed}(s)$ = number of passing tests that execute s

total_failed = total number of failing tests

total_passed = total number of passing tests

Tarantula suspiciousness:

suspiciousness(s) = use the Tarantula formula

The minimum is to compute suspiciousness based on Tarantula. You can use additional SBFL suspiciousness formulas.

The LLM should receive only the ranking, not the ground-truth faulty line.

5. Metrics

You must report at least the following metrics.

Metric	Required
Top-1 exact-line accuracy	Yes
Top-3 accuracy	Yes
Region accuracy	Yes
Mean reciprocal rank (MRR)	Yes
Invalid-output rate	Yes

You should report the metrics:

- per LLM
- per information condition
- overall
- optionally, per bug type

6. Validation of Your Own Infrastructure

A central part of this assignment is that you are not only running an experiment — you are building software that runs an experiment.

You are required to validate the correctness of your own experimental infrastructure.

At minimum, you must test:

- mutation-generation code
- SBFL computation code
- result-computation code
- output parsing code
- metrics-computation code
- HTML-generation scripts
- coverage aggregation code

Your infrastructure tests should follow testing best practices discussed in class.

You should aim for tests that are:

- Fast
- Independent
- Repeatable
- Self-checking

You should use:

- clear test names
- Arrange–Act–Assert structure
- meaningful assertions

You are encouraged to measure test-suite quality for your own infrastructure code using:

- line coverage
- branch coverage
- method/function coverage

You should not treat coverage as proof of correctness, but you should use it as evidence that important parts of your experiment code were exercised.

7. Automation Requirements

Your artifact should automate at least:

- test execution
- coverage collection
- Tarantula computation
- prompt generation
- OpenRouter querying
- saving raw LLM outputs

- output parsing
 - metric computation
 - infrastructure testing
 - result aggregation
 - plot generation
-

8. HTML Website Requirements

Your final report must be a professional static HTML website.

The website should include the following sections.

Overview

- project goals
- benchmark used
- summary of findings

Dataset

- HumanEval source
- number of methods
- mutation strategy
- examples of bugs
- dataset table

Experimental Design

- models
- information conditions
- prompt templates
- metrics
- procedure

Validation of the Experiment Infrastructure

- tests for mutation-generation code
- tests for SBFL computation
- tests for scoring and result-generation code
- number of infrastructure tests
- coverage or other test-quality measures
- examples of bugs found during validation
- remaining limitations

Results

- tables comparing models and conditions
- plots and graphs
- Top-1 accuracy
- Top-3 accuracy
- invalid-output rate
- interpretation of findings

Qualitative Analysis

- examples where tests helped
- examples where Tarantula helped
- examples where Tarantula misled the model
- examples of plausible but wrong explanations

Threats to Validity

Discuss:

- benchmark representativeness
- equivalent mutants
- oracle limitations
- LLM nondeterminism
- prompt sensitivity
- measurement bias
- API failures and rate limits

Reproducibility

Include:

- exact commands
- environment setup
- Java version
- dependency setup
- OpenRouter setup
- reproduction instructions

Use of AI Tools

You must document:

- which AI tools were used
- how they were used
- how outputs were validated

- mistakes found and corrected
-

9. Weekly Milestones

Week 1 — Dataset and Experiment Planning

Deliverables

- initial dataset selection
- successful python testing setup
- mutation plan
- experiment design draft
- infrastructure-validation plan
- repository skeleton
- README draft

Week 2 — Mutants, Tests, Coverage, and SBFL

Deliverables

- validated buggy methods
- coverage collection
- Tarantula rankings
- updated prompt templates
- first infrastructure tests
- partial automation scripts

Week 3 — LLM Execution and Result Computation

Deliverables

- OpenRouter integration
- generated prompts
- raw LLM outputs
- parsing and scoring scripts
- tests for scoring/result-generation code
- preliminary results
- initial qualitative examples

Week 4 — Final Artifact and Website

Deliverables

- complete website
- final reproducibility artifact

- final results
- final plots
- final analysis
- validation summary
- threats-to-validity discussion
- AI-tool-use discussion
- cleaned repository
- finalized documentation

10. Grading Rubric

Component	Points
Dataset construction and mutant validation	15
Experimental design and controls	10
OpenRouter integration and automation	15
Test execution, coverage, and Tarantula computation	10
Validation and testing of experiment infrastructure	15
Metrics, scoring, and result correctness	10
Analysis and interpretation	10
HTML website quality	5
Reproducibility artifact	3
Video presentation quality	5
Threats to validity and AI-tool discussion	2

11. Project Presentation Video

In addition to the website and reproducibility artifact, you must submit a short project presentation video. The purpose of the video is to provide a concise overview of your work, methodology, results, and conclusions.

11.1 Requirements

The video must:

- be between **5 and 10 minutes** long
- use presentation slides in English only
- include screenshots from different parts of your project
- include narration by the student(s) in English or Hebrew
- not show the student(s) on camera
- be submitted in a common video format such as MP4

The video should consist primarily of:

- slides
- screenshots
- figures
- tables
- plots
- diagrams

The student's voice must be recorded as a voiceover throughout the presentation.

11.2 Suggested Structure

Introduction

- project goals
- overview of the main goals of the assignment

Data Collection and Evaluation Setup

- Overview of the methods and the mutations
- Overview of the tests
- LLMs used
- Data collection

Results

- Tables and figures showing the results of the experiments
- Answers to the research questions

Conclusions

- key findings
- lessons learned
- limitations
- future improvements

11.3 Required Screenshots

The video must include screenshots from several parts of the project, such as:

- repository structure
- example methods and tests
- SBFL reports
- test coverage of your own code
- HTML website
- selected source-code examples

You are encouraged to annotate screenshots to help explain important observations.

11.4 Evaluation Criteria

The video will be evaluated based on:

- clarity of presentation
- technical accuracy
- quality of explanation
- effective use of slides and screenshots
- discussion of results
- organization and pacing
- professionalism

12. Use of AI-Based Tools

You are encouraged to use AI-based tools during this assignment.

You may use them for:

- generating code
- writing scripts
- debugging
- generating mutants
- improving prompts
- creating plots
- improving website quality
- improving documentation

However, you remain responsible for correctness.

Your final website must include a section titled “Use of AI Tools” explaining:

- which tools were used
 - what they were used for
 - how outputs were validated
 - what mistakes were discovered and corrected
-

13. Submission Requirements

Submit:

1. Source-code repository
2. Reproducibility artifact
3. Static HTML website
4. Project presentation video (5–10 minutes)

Do not submit API keys or private credentials.

Your repository must include clear reproduction instructions.

14. Final Notes

This assignment is intended to simulate a realistic empirical software engineering project involving:

- empirical software engineering
- software testing
- fault localization
- experimental methodology
- reproducibility engineering
- infrastructure validation

You are encouraged to:

- automate as much as possible
- justify design decisions
- critically analyze your results
- discuss limitations honestly
- validate your own infrastructure carefully